

## BAB 7: RECYCLERVIEW UNTUK DATA LIST

### Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa diharapkan mampu:

1. Memahami konsep dan keunggulan `RecyclerView` dibandingkan `ListView`.
2. Menjelaskan peran dari `Adapter`, `ViewHolder`, dan `LayoutManager`.
3. Menerapkan `RecyclerView` untuk menampilkan sekumpulan data dalam bentuk daftar (list).
4. Membuat aplikasi daftar kontak sederhana menggunakan `RecyclerView`.

### 7.1 Konsep RecyclerView

Menampilkan data dalam bentuk daftar adalah kebutuhan umum dalam aplikasi mobile. Android menyediakan `RecyclerView` sebagai komponen standar untuk tugas ini. `RecyclerView` adalah versi yang lebih canggih dan fleksibel dari `ListView` yang sudah lama ada.

**Keunggulan `RecyclerView`:**

- **Reusing Views:** `RecyclerView` mendaur ulang (recycle) item-item yang sudah tidak terlihat di layar untuk menampilkan data baru yang masuk ke layar. Ini secara drastis meningkatkan performa dan mengurangi konsumsi memori, terutama untuk daftar yang sangat panjang.
- **Decoupling:** `RecyclerView` memisahkan tugas menjadi beberapa komponen, membuat kode lebih terorganisir dan mudah dikelola.

**Komponen Penting dalam `RecyclerView`:**

1. `RecyclerView`: Widget UI itu sendiri yang ditempatkan di layout XML. Tugasnya hanya menampilkan item.
2. `LayoutManager`: Bertanggung jawab untuk mengatur posisi dan ukuran setiap item di layar. Android menyediakan `LinearLayoutManager` (untuk daftar vertikal/horizontal), `GridLayoutManager` (untuk grid), dan `StaggeredGridLayoutManager`.
3. `Adapter`: Jembatan antara sumber data (misalnya, `ArrayList`) dan `RecyclerView`. Tugasnya adalah membuat `ViewHolder` dan mengisi data ke dalam `ViewHolder` berdasarkan posisi item.
4. `ViewHolder`: Objek yang menyimpan referensi ke View untuk satu item di daftar (misalnya, satu `TextView` dan `ImageView` dalam satu baris). Dengan menyimpan referensi ini, kita tidak perlu memanggil `findViewById()` berulang kali, yang meningkatkan efisiensi.

### 7.2 Implementasi Praktik: Aplikasi Daftar Kontak

Kita akan membuat aplikasi yang menampilkan daftar nama dan nomor telepon kontak.

#### Langkah 1: Tambahkan Dependensi

Pastikan dependensi `RecyclerView` sudah ada di `build.gradle.kts` (Module :app). Jika tidak, tambahkan:

```
dependencies {  
    // ...  
    implementation("androidx.recyclerview:recyclerview:1.3.2") // Gunakan versi  
    terbaru  
}
```

### Langkah 2: Buat Model Data ( `Contact.kt` )

Buat file Kotlin baru untuk mendefinisikan struktur data kontak.

```
// Contact.kt  
data class Contact(  
    val name: String,  
    val phoneNumber: String  
)
```

### Langkah 3: Buat Layout untuk Satu Item ( `item_contact.xml` )

Buat file layout baru di `res/layout/item_contact.xml`. Ini adalah tampilan untuk satu baris di daftar kontak.

```
<?xml version="1.0" encoding="utf-8"?>  
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:orientation="vertical"  
    android:padding="16dp">  
  
    <TextView  
        android:id="@+id/tvName"  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:textSize="18sp"
```

```

        android:textStyle="bold"
        tools:text="John Doe" />

<TextView
    android:id="@+id/tvPhoneNumber"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textSize="14sp"
    tools:text="08123456789" />

</LinearLayout>

```

#### Langkah 4: Buat Adapter dan ViewHolder ( `ContactAdapter.kt` )

Buat file Kotlin baru untuk adapter.

```

// ContactAdapter.kt
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView

class ContactAdapter(private val contactList: List<Contact>) :
    RecyclerView.Adapter<ContactAdapter.ContactViewHolder>() {

    // ViewHolder: Menyimpan referensi ke View dalam satu item layout
    class ContactViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val tvName: TextView = itemView.findViewById(R.id.tvName)
        val tvPhoneNumber: TextView = itemView.findViewById(R.id.tvPhoneNumber)
    }

    // onCreateViewHolder: Dipanggil saat RecyclerView perlu membuat ViewHolder
    // baru
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
        ContactViewHolder {
        val itemView = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_contact, parent, false)
        return ContactViewHolder(itemView)
    }
}

```

```
// onBindViewHolder: Dipanggil untuk menghubungkan data dengan ViewHolder pada
posisi tertentu
override fun onBindViewHolder(holder: ContactViewHolder, position: Int) {
    val currentContact = contactList[position]
    holder.tvName.text = currentContact.name
    holder.tvPhoneNumber.text = currentContact.phoneNumber
}

// getItemCount: Mengembalikan jumlah total item dalam data
override fun getItemCount(): Int {
    return contactList.size
}
}
```

#### Langkah 5: Tambahkan RecyclerView di Layout Utama ( activity\_main.xml )

```
<!-- activity_main.xml -->
<androidx.constraintlayout.widget.ConstraintLayout ...>
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvContacts"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        tools:listitem="@layout/item_contact" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

#### Langkah 6: Atur RecyclerView di Activity ( MainActivity.kt )

```
// MainActivity.kt
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
```

```

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rvContacts = findViewById<RecyclerView>(R.id.rvContacts)

        // 1. Siapkan data (contoh data statis)
        val contactList = listOf(
            Contact("Alice", "0811111111"),
            Contact("Bob", "0822222222"),
            Contact("Charlie", "0833333333"),
            Contact("Diana", "0844444444"),
            Contact("Eve", "0855555555")
        )

        // 2. Buat instance adapter
        val adapter = ContactAdapter(contactList)

        // 3. Atur LayoutManager
        rvContacts.layoutManager = LinearLayoutManager(this)

        // 4. Atur adapter pada RecyclerView
        rvContacts.adapter = adapter
    }
}

```

## 7.3 Studi Kasus Latihan Praktikum (Mini-Proyek)

### Studi Kasus: Aplikasi Daftar Tugas (To-Do List)

1. **Buat Model Data:** `Task` dengan properti `title: String` dan `isCompleted: Boolean`.
2. **Buat Layout Item:** `item_task.xml` yang berisi `CheckBox` dan `TextView` untuk judul tugas.
3. **Buat Adapter:** `TaskAdapter` yang menampilkan daftar tugas. Saat `CheckBox` dicentang, ubah gaya teks `TextView` (misalnya, dicoret) dan update status `isCompleted` di dalam model data.
4. **Buat Activity Utama:** Tampilkan `RecyclerView` dengan daftar tugas awal (misalnya, "Belajar Kotlin", "Buat modul", "Olahraga").
5. **Tantangan Tambahan:** Tambahkan `FloatingActionButton` yang ketika diklik, menampilkan `Dialog` atau `Activity` baru untuk menambahkan tugas baru ke dalam daftar. (Ini akan membutuhkan konsep notifikasi perubahan data pada adapter, yang bisa dipelajari lebih lanjut).